Université djillali liabes

Faculté des Sciences exactes
Département d'Informatique

TRAVAUX PRATIQUES N°1

Cryptographie et Sécurité

Algorithme d'Euclide étendu

Inversion modulaire

Chiffrement affine

Année universitaire :	2025-2026
Enseignant :	sahraoui mohamed
Groupe :	Groupe 1 isi

ÉTUDIANTE :

meroufel widad louiza

0.1 Description mathématique

1 Algorithme d'Euclide étendu

L'algorithme d'Euclide étendu permet de calculer le PGCD de deux entiers a et b ainsi que deux entiers u et v tels que :

$$au + bv = \gcd(a, b)$$

À chaque étape, l'algorithme préserve deux relations fondamentales :

Invariant 1 : r = a $\cdot u + b \cdot v$

Invariant 2 : r' = a $\cdot u' + b \cdot v'$

où :

- r : reste courant
- r' : reste suivant
- u, v : coefficients pour le reste courant
- u', v' : coefficients pour le reste suivant
- a, b : entiers initiaux

Cet algorithme est une extension de l'algorithme d'Euclide classique. Il ne calcule pas seulement le PGCD, mais aussi les coefficients de Bézout. Il est utilisé notamment dans la cryptographie RSA pour calculer l'inverse modulaire.

2 Algorithme d'Euclide étendu

```
class BigIntMath:
    # Définition d'une classe pour les opérations mathématiques
    # sur les grands entiers

    @staticmethod
    # Méthode statique : peut être appelée sans créer d'objet

    def extendedGCD(a, b):
        # Fonction qui calcule :
        # - le PGCD(a,b)
        # - les coefficients de Bézout u et v
        # tels que : a*u + b*v = pgcd(a,b)

        u0, u1 = 1, 0
        # Initialisation des coefficients u

        v0, v1 = 0, 1
        # Initialisation des coefficients v

        while b != 0:
            # Boucle principale de l'algorithme d'Euclide
```

```
            q = a // b
            # Calcul du quotient de la division entière

            a, b = b, a - q * b
            # Mise à jour des valeurs (reste de division)

            u0, u1 = u1, u0 - q * u1
            # Mise à jour des coefficients u

            v0, v1 = v1, v0 - q * v1
            # Mise à jour des coefficients v

        return a, u0, v0
        # a contient le PGCD
        # u0 et v0 sont les coefficients de Bézout
```

3 algorithme d'Inversion modulaire

```
@staticmethod
# Déclaration d'une méthode statique utilisable sans instancier la classe

def modInverse(a, n):
    # Cette fonction calcule l'inverse modulaire de a modulo n

    pgcd, u, _ = BigIntMath.extendedGCD(a, n)
    # Exécution de l'algorithme d'Euclide étendu
    # pgcd : plus grand commun diviseur de a et n
    # u : coefficient permettant d'obtenir l'inverse modulaire
    # _ : coefficient non utilisé dans cette fonction

    if pgcd != 1:
        # Si a et n ne sont pas premiers entre eux,
        # l'inverse modulaire n'existe pas
        return None
    return u % n
    # Renvoie l'inverse modulaire sous forme positive
    # en appliquant l'opération modulo n
```

4 le chiffrement affine

```
class AffineCipher:
    # Classe implémentant le chiffrement affine

    """Chiffrement affine utilisant l'inversion modulaire"""
```

```
def __init__(self, a, b, n=26):
    # Constructeur de la classe
    # a : coefficient multiplicatif
    # b : coefficient additif
    # n : taille de l'alphabet (26 lettres)

    self.a = a
    # Stocke la clé multiplicative

    self.b = b
    # Stocke la clé additive

    self.n = n
    # Modulo utilisé pour les calculs

    self.a_inv = BigIntMath.modInverse(a, n)
    # Calcul de l'inverse modulaire de a modulo n

    if self.a_inv is None:
        # Vérifie si l'inverse modulaire existe
        raise ValueError(f"a={a} n'est pas inversible modulo {n}")
        # Génère une erreur si a n'est pas inversible

def encrypt(self, m):
    # Fonction de chiffrement d'un symbole numérique
    # E(m) = (a*m + b) mod n
    return (self.a * m + self.b) % self.n

def decrypt(self, c):
    # Fonction de déchiffrement
    # D(c) = a^{-1}(c - b) mod n
    return (self.a_inv * (c - self.b)) % self.n

def encrypt_text(self, text):
    # Chiffrement d'une chaîne de caractères complète

    result = ""
    # Chaîne contenant le texte chiffré

    for char in text.upper():
        # Parcourt chaque caractère du texte
        # Conversion en majuscules

        if 'A' <= char <= 'Z':
            # Vérifie si le caractère est une lettre

            m = ord(char) - ord('A')
            # Transformation lettre → nombre (A=0,...,Z=25)
```

```
            c = self.encrypt(m)
            # Application du chiffrement affine

            result += chr(c + ord('A'))
            # Transformation nombre → lettre

        else:
            result += char
            # Conserve espaces et symboles inchangés

    return result
    # Retourne le texte chiffré

def decrypt_text(self, text):
    # Déchiffrement d'un texte complet

    result = ""
    # Chaîne contenant le texte déchiffré

    for char in text.upper():
        # Parcourt chaque caractère

        if 'A' <= char <= 'Z':
            # Vérifie si c'est une lettre

            c = ord(char) - ord('A')
            # Conversion lettre → nombre

            m = self.decrypt(c)
            # Application du déchiffrement affine

            result += chr(m + ord('A'))
            # Conversion nombre → lettre

        else:
            result += char
            # Les caractères spéciaux restent identiques

    return result
    # Retourne le texte déchiffré
```

La classe AffineCipher implémente le chiffrement affine défini par :

$$E(m) = (am + b)n$$

et le déchiffrement :

$$D(c) = a^{-1}(c - b)n$$

où a^{-1} représente l'inverse modulaire de a modulo n, calculé à l'aide de l'algorithme d'Euclide étendu.

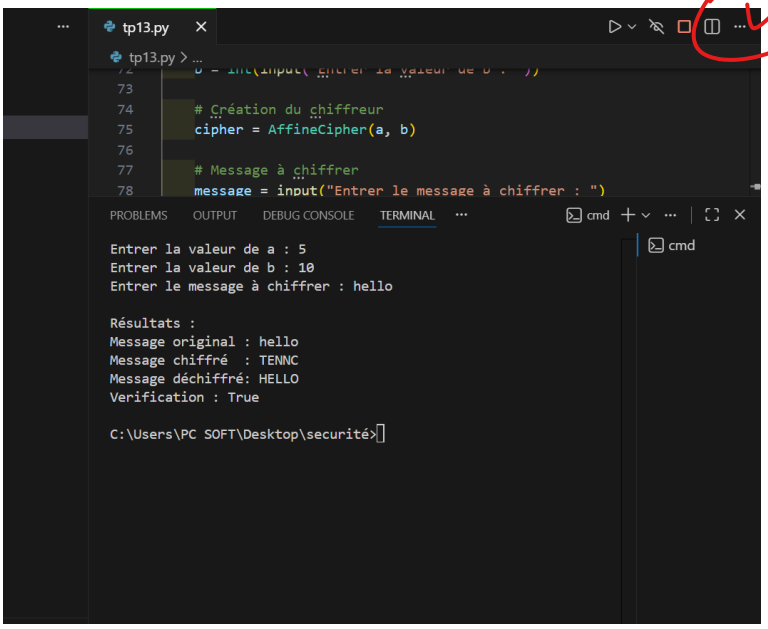

FIGURE 1 – les résultats d'éxecution

5 le lien entre Euclide étendu, inversion modulaire et chiffrement affine :

L'algorithme d'Euclide étendu établit le lien fondamental entre ces trois concepts : pour deux entiers a et n, il calcule les coefficients u et v tels que a·u + n·v = pgcd(a, n). Lorsque pgcd(a, n) = 1, condition requise pour le chiffrement affine, cette équation devient a·u + n·v = 1. En réduisant modulo n, on obtient a·u 1 (mod n), ce qui signifie que u mod n est exactement l'inverse modulaire a^{1}. Le chiffrement affine utilise directement cet inverse pour le déchiffrement : D(c) = a^{1}·(c - b) mod n. Ainsi, Euclide étendu permet à la fois de vérifier l'inversibilité de a, de calculer son inverse, et de garantir le fonctionnement du chiffrement affine. Sans cet algorithme, le déchiffrement serait impossible.